



BSc (Hons) in **Information Technology**  
**Specialized in AI**

**IT3012 : Intelligent Agents**

Year 3 · Semester 1 · 2026 Semester 2

**Faculty of Computing**

## Practical 01

**Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

### Part 1: Code Analysis & PEAS Evaluation

#### Task 0 : Setting up and getting the starter Code

1. Go to the [Git Hub Repo](#). Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

#### Task 1.1: The Environment State (`__init__`)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P – Performance Measure: Defines how the agent's success is evaluated.  
E – Environment: The external world in which the agent operates, A – Actuators: The mechanisms or actions the agent uses to interact with the environment, S – Sensors: The devices or methods the agent uses to perceive the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

`self.width` and `self.height` — define the size of the grid world, `self.walls` — the locations of obstacles in the environment, `self.food_positions` — the locations of all food items that can be collected, `self.opponents` — the positions of the adversarial

**3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.**

The baseline environment is multi-Agent because it includes the main agent (`self.agent_pos`) and one or more opponent agents (`self.opponents`) that coexist and interact within the same environment.

## Task 1.2: The Perception Subsystem (`get_percept`)

**4. (Remember) Define what a "percept sequence" is according to Lecture 01.**

The complete history of all the percepts (observations) an agent has received from the environment since it began operating.

**5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?**

The `get_percept()` method represents the Sensors (Percepts) component of the PEAS framework because it provides the observations that the agent receives from the environment.

**6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.**

The environment is Fully Observable because the agent receives complete information about its own state and all opponents' positions, along with other relevant status information. It does not have hidden or missing information about the

## Task 1.3: Action Execution (`execute_action`)

### 7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

The code is updating the Performance Measure (P) because it deducts 5 points from `self.score` when the agent hits a wall, penalizing undesirable behavior.

### 8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

It is structurally important because the Performance Measure (P) in PEAS is meant to evaluate the agent's success in achieving goals within the external environment, not how much internal computation or effort the agent uses.

## Part 2: Guided Modification & Deep Theory Mapping

Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

### Step 2.1: Extending the Environment Initialization (`__init__`)

- Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

### 9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally **hide** this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment becomes Partially Observable because the true environment state includes toxic traps, but the agent's sensors no longer provide that information. The agent must make decisions with incomplete knowledge of the world.

### Step 2.2: Updating the Perception Subsystem (`get_percept`)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Adding smells\_toxin gives the agent early information about dangerous cells. This helps the agent avoid toxic traps, reduce penalties, and choose safer actions. By considering this sensor, the agent can maximize expected utility by balancing

### Step 2.3: Modifying Action Execution & Visual Rendering (`execute_action` & `draw_grid`)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The classic Vacuum World exploit was when an agent learned to maximize its performance metric by removing dirt from only one square repeatedly or doing unnecessary actions, instead of actually cleaning the environment effectively. It showed that a faulty metric can encourage unintended behavior.

Finalize and Lock Lab 01 Analytical Evaluation



**FACULTY OF COMPUTING**